1. To implement the equivalence of /co-area-drivers in a relational database we need to do Self-Joins on the Trips table. This would filter the first instance for the input of driver_id, join it to the second instance on the dropoff area, and exclude the original queried driver from the matches. We will group the resulting records by the matched drivers, count the unique number of shared dropoff areas for each driver and sort in descending order. This requires expensive index lookups or full table scans making the cost grow with total table size rather than neighborhood size.
2. Traversal is more efficient in Neo4j because it uses index-free adjacency. So each node physically stores direct pointers to its neighbors and relationships. Traversing from a driver to an area to another driver is O(1) due to pointers. Relational databases have O(logN) due to global B-Tree index lookups to match foreign keys for each step of a join. So it is more natural for /co-area-drivers queuers as the depth of the traversal has no effect on Neo4j while having large impacts for relational database queries.
3. Neo4j would underperform a relational database on global aggregate queries across the entire dataset. For example, if we wanted to compute the average fare across all the trips in the dataset a relational database can just do a full-scan columnar aggregation, but Neo4j needs to traverse every relationship. So bulk analytics requires way more overhead for Neo4j making it slower and inefficient compared to relational databases.